

Frontend Intern Assignment

Fleet Management Dashboard | React + Supabase | AI-assisted engineering

Role	Frontend Engineer Intern
Core stack	React + TypeScript frontend, Supabase Auth + Postgres backend.
Assignment link	https://forms.gle/byk2ZDWnM1gEfyXA8
Submission	Upload one ZIP through the provided link. Do not upload the assignment to GitHub; use the submission link only. ZIP size must not exceed 100 MB.
Objective: Design and build a small but credible fleet management product prototype using React and Supabase. The app should show how you think about connected vehicle data, operational dashboards, and AI-assisted frontend engineering.	

Product Scenario

Bytebeam works with connected devices and vehicle telemetry. For this assignment, assume an internal operations team needs a lightweight fleet dashboard to monitor vehicles, review recent routes, and spot obvious vehicle-health or usage patterns.

You do not need real devices, real GPS ingestion, or production telemetry. Use realistic seeded data and make enough product decisions that we can understand how you approach an ambiguous frontend problem.

Product Brief

- The app should have a login experience and a protected fleet dashboard experience.
- The dashboard should help an operations user understand where vehicles are, what they have been doing, and whether anything needs attention.
- We will evaluate how well you show fleet and trip data on screen: clarity, prioritization, layout, interaction design, and whether the dashboard helps answer real operations questions.
- A map should be central to the experience. It should be possible to inspect a vehicle's movement history or route in a way that is useful to an operations user.
- Show a small set of high-level vehicle or trip parameters. You decide which parameters matter and how to present them.
- Avoid building a marketing page. Build the product surface an internal user would actually operate.

Sample parameters to consider: distance, duration, average or maximum speed, idle time, halt count, overspeed events, start/end location, last seen time, fuel/battery, ignition, and driver or vehicle status. These are examples, not a checklist; choose what fits your product.

Backend And Data Requirements

Use Supabase as the backend. You should design the data model yourself. We intentionally are not providing a schema because we want to see how you think about the domain and how well you model fleet and trip data in the backend.

Your backend should contain enough persisted data for us to evaluate the dashboard as a working product prototype, not just a static mock. Use sample data that feels plausible for a fleet-management system.

- Include setup notes so we can understand and recreate your Supabase project structure.
- Briefly explain the main backend decisions you made and any tradeoffs.

Technical Expectations

- Use React + TypeScript for the frontend.
- Use Supabase for authentication and backend data.
- Use a real map library or map integration that fits your design.
- Make the project easy to install, run, and review. Include README setup instructions.
- A deployed demo is expected. Vercel, Netlify, Render, or another reasonable host is fine.
- Quality matters more than feature count. A small coherent product is better than a large unfinished scaffold.

AI Usage Is Encouraged

You may use AI tools such as Cursor, Claude, ChatGPT, Copilot, Lovable, Bolt, v0, or similar tools. AI-assisted development is not a negative signal. We want you to use AI well, and your chat logs and interview discussion should make that visible.

What we are evaluating: We want to distinguish thoughtful AI-assisted engineering from unreviewed vibe-coding. You should be able to explain the architecture, data model, library choices, generated code, tradeoffs, and bugs you found or prevented.

Required AI Chat Logs

- Use AI tools that support exporting or sharing chat logs, and include those logs or links in your submission ZIP.
- In your README, list the AI tools you used and where they helped.

Deliverables

Deliverable	Required	Notes
Submission ZIP	Yes	Upload one ZIP through https://forms.gle/byk2ZDWnM1gEfyXA8 . Include source code, one README, and AI chat logs. Do not upload to GitHub. ZIP size must not exceed 100 MB.
Source code	Yes	Include the project source files. Do not include node_modules, build output, cache folders, or other unnecessary bulky files.
Single README	Yes	Include one README with the live app URL, demo login credentials if needed, backend setup notes, walkthrough link if present, AI tools note, and any other notes requested here.
AI chat logs	Yes	Include exported chat logs, or a file containing shared chat-log links, in the same ZIP.

Interview Follow-Up

If shortlisted, expect a short technical discussion around the code you submitted. We may ask you to:

- Walk through the product decisions you made and why.
- Explain your Supabase model and backend decisions.
- Discuss how AI was used in the submitted code.
- Make a small live change to the dashboard.
- Discuss what you would improve if this became a production fleet dashboard.

Optional Stretch Work

If you have extra time, add one thoughtful extension that makes sense for your product direction. Do not add features just to make the app look larger.

Submission Notes

- Prioritize a coherent, working slice over a large unfinished app.
- Put the source code, single README, and AI chat logs into one ZIP, upload it through the form, and keep it under 100 MB.
- Before uploading, remove node_modules, build output, cache folders, screenshots/videos not needed for review, and anything else that unnecessarily increases upload size.
- Templates and AI-generated scaffolds are allowed, but you must adapt and understand them.
- Make the app easy for us to run and evaluate without a long back-and-forth.